Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Giovanni Denaro***
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

I Prova di Ingegneria del Software

29 Aprile 2002

Cognome

Nome

Matricola

Sezione (segnarne una) Denaro ☐, Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno presi in considerazione.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 3h.
6. Punteggio totale a disposizione: 40 punti nominali (10 per ogni esercizio), corrispondenti a 13/30 disponibili per la prima prova. La sufficienza corrisponde a circa 18 punti.

Esercizio 1:
Parte (a):
Si considerino le seguenti classi Punto2D e Punto3D, dichiarate in file separati Punto2D.java, Punto3D.java del package Geometria. Completare opportunamente le parti con i puntini, aggiungendo, ove necessario, modificatori di visibilità, dichiarazioni e/o implementazioni di metodi, ecc. Si trascuri l'implementazione di altri metodi oltre a distanza, equals e i costruttori (come ad esempio toString() e repOk()).

```
package Geometria;.....
public..... class Punto2D {
//OVERVIEW: Tipo immutabile che rappresenta un punto in uno spazio
//euclideo a due dimensioni.
//Un punto tipico è individuato dalle sue coordinate cartesiane (x,y).
//AF(c) = (c.x, c.y)
//RI: true
//rep: basato su coordinate cartesiane
protected..... float x, y; //protected perché x y usate da Punto3D
    public Punto2D (float x, float y) {
        this.x = x; this.y=y; .....//aggiriamo mascheramento
    }
    public float distanza (Punto2D p) {
        return (float)Math.sqrt((p.x-x) * (p.x-x) +(p.y-y) * (p.y-y));
    }
    public boolean equals(Punto2D p) {
        return(x == p.x && y == p.y);
    }
}
. package Geometria;.....
. public..... class Punto3D extends Punto2D {
//OVERVIEW: Tipo immutabile che rappresenta un punto in uno spazio
//euclideo a tre dimensioni.
//Un punto tipico è individuato dalle sue coordinate cartesiane (x,y,z).

protected..... float z;
    public Punto3D (float x, float y, float z) {
        super(x,y);  this.z=z;.....
// usare super(x,y) per inizializzare le componenti ereditate: altrimenti
// chiamato costruttore di default di Punto2D, che non esiste!
    }
//N.B. il metodo distanza che segue non è una ridefinizione di quello della
//classe  Punto2D, in quanto i parametri hanno tipo diverso. Pertanto il
//metodo della classe Punto2D continua ad essere usabile, in quanto
//ereditato. Il metodo che segue dunque semplicemente realizza overloading.

    public float distanza (Punto3D p) {
        return (float)Math.sqrt((p.x-x) * (p.x-x) +(p.y-y) * (p.y-y) +(p.z-z) * (p.z-z));
    }
    public boolean equals (Punto2D p )  {
        if (p instanceof Punto3D) return equals((Punto3D) p);
        return super.equals(p);
    }
    public boolean equals (Punto3D p )  {
```

```
// il metodo è chiamato da equals(Punto2D): se assente, chiamata di
// equals con un Punto3D causa ricorsione infinita (che provoca Stack Overflow)
        if (z==p.z) return super.equals(p);
//NB:si puo' anche scrivere x == p.x &&y==p.y ecc. al posto di super.equals
        return false;
    }
.....
.....
.....
}
```

Alcune osservazioni:
L'esercizio precedente non è un esempio di buona progettazione, ma è servito solo allo scopo di verificare la vostra comprensione dei meccanismi della programmazione object oriented. L'ereditarietà è stata infatti usata in maniera impropria. Un punto a 3 dimensioni NON È un punto a 2 dimensioni; non ne conserva le proprietà.
Un'organizzazione migliore per questo stesso problema è quella dell'esercizio 2.
Si osservi che in generale sarebbe stato meglio che in Punto2D fosse ridefinita anche la equals(Object):

```
public boolean equals (Object o) {
        if (o instanceof Punto2D) return equals((Punto2D) o);
        return false;}
```

In questo modo si evitano problemi di casting quando si chiama equals (v. esercizio 4, parte a). Ovviamente questo non era richiesto per l'esame.

Esercizio 1, Parte (b)
In un file separato di nome PuntoUser.java, contenuto in un altro package, è riportata la seguente classe (NB: si rammenta che import permette di importare una classe e utilizzarne i metodi pubblici senza dot notation):

```
import Geometria.Punto2D, Geometria.Punto3D;
public class PuntoUser {
  public static void main(String args[]) {
    Punto2D p1,p2;
    Punto3D p3;
    p1 = new Punto2D(3,7);
    p2 = new Punto3D(3,7, 4); OK, assegnamento polimorfico
    System.out.println(p1.distanza(p2));   0.0: Chiamato metodo di Punto2D
    System.out.println(p2.distanza(p1));   0.0 Punto2D
    p3 = new Punto3D(6,7, 5);
    System.out.println(p2.distanza(p3));   3.0 Punto2D
    System.out.println(p3.distanza(p1));   3.0 Punto2D
    System.out.println(p1.distanza(p3));   3.0 Punto2D

    Punto3D p4 = new Punto3D(6,1, 5);
    System.out.println(p3.distanza(p4));   6.0 Punto3D
    System.out.println(p1.x); scorretto: x è protected
    p1 = p2; OK, assegnamento polimorfico
    p3 = new Punto3D(); scorretto: manca costruttore di default
    p3 =  p1; scorretto: tipo statico di p1 (Punto2D) non è sottotipo del tipo statico
di  p3. : correggere in p3 = (Punto3D) p1; perché  tipo dinamico di p1 è
compatibile con Punto3D, che è il tipo statico di p3.
    p3 = p2; scorretto: tipo statico di p2 non è sottotipo del tipo di p3.
Correggere in p3 = (Punto3D) p2; perché  tipo dinamico di p2 è compatibile con
Punto3D, che è il tipo statico di p3.
  }
}
```

Tutte le linee sono sintatticamente corrette, ma ci possono essere errori legati alla visibilità e al controllo statico dei tipi. Segnare le linee non corrette, spiegandone brevemente il motivo, e proporne, se possibile, una correzione. Si ricorda che il metodo statico System.out.println() stampa in ouput il valore dell'espressione passata come argomento. Per ogni istruzione di stampa, riportare a fianco il risultato stampato durante l'esecuzione (sotto l'ipotesi di eliminazione dal codice di tutte le linee scorrette). Per ogni chiamata del metodo distanza indicare se viene chiamato il metodo definito in Punto2D o in Punto3D.

Esercizio 2,
Parte (a)
Definire un'interfaccia Punto, che rappresenta un punto in un generico spazio euclideo. Non vi è alcuna indicazione del numero di coordinate, ma deve essere previsto un metodo per la distanza.

public.....interface Punto
{
public.....float distanza (Punto p.....);
.....
.....
}

Riscrivere Punto2D e Punto3D in modo che siano implementazioni di Punto, e in modo che Punto3D non sia erede di Punto2D. Per brevità si trascurino i metodi equals, la Overview, AF e RI.

.....class Punto2D.....
.....
.....
.....
.....
.....
}

.....class Punto3D…….....
.....
.....
.....
.....
.....
}

Soluzione:

```
public interface Punto
{
    public float distanza (Punto p);
}

public class Punto2D implements Punto {
    private float x, y; //NB: anche protected accettabile
    public Punto2D (float x, float y) {
        this.x = x; this.y=y;
    }
    public float distanza (Punto p) {
        if (!(p instanceof Punto2D)) throw new ClassCastException();
//NB: la linea precedente si poteva anche omettere perche' ClassCastException
//(eccezione unchecked) può essere lanciata dal cast a Punto2D
        Punto2D p1 = (Punto2D) p;
          return (float) Math.sqrt((p1.x-x) * (p1.x-x) +(p1.y-y) * (p1.y-y));
     }
}
```



```
class Punto3D implements Punto {
    private float x, y,z; //NB: anche protected accettabile
    public Punto3D (float x, float y, float z) {
        this.x = x; this.y=y; this.z=z;
     }
    public float distanza (Punto p) {
        if (!(p instanceof Punto3D)) throw new ClassCastException();
 //NB: la linea precedente si poteva anche omettere...
        Punto3D p1 = (Punto3D) p;
        return (float) Math.sqrt((p1.x-x) * (p1.x-x) +(p1.y-y) * (p1.y-y) +(p1.z-z) *
(p1.z-z));
    }
}
```

Esercizio 2. Parte (b)

Si ipotizzi che l'interfaccia Punto e le classi Punto2D e Punto3D siano incluse in un package Punti. In un file separato di nome PuntoUser2.java, contenuto in un altro package, è riportata la seguente classe:

```
import Punti.*;
public class PuntoUser2 {
  public static void main(String args[]) {
    Punto p1,p2, p3;
    p1 = new Punto2D(3,7);
    p2 = new Punto3D(3,7, 9);
    p3 = new Punto3D(4,7,9);
    System.out.println(p2.distanza(p3));
    System.out.println(p1.distanza(p2));
    System.out.println(p2.distanza(p1));
  }
}
```

Qualè il risultato dell'esecuzione del codice?

Stampa 1, poi lancia eccezione ClassCastException in quanto p2 ha tipo apparente Punto compatibile con p1.distanza(Punto), ma tipo effettivo Punto3D è invece incompatibile e provoca eccezione in metodo distanza.
Se anche si eliminasse la seconda print, anche la terza print lancerebbe eccezione.

Esercizio 3
Sono date le seguenti specifiche della classe Segmento e della classe Spezzata, che sono dichiarate nel package Geometria. Le classi usano la classe Punto2D dell'esercizio 1 e una classe DuplicatePointException per le eccezioni, da non definire.

```
public class Segmento {
//OVERVIEW: un segmento è una coppia ordinata di punti distinti. Tipo immutabile.
// Un segmento tipico è una coppia (inizio, fine)  con:
//inizio!= fine, inizio e fine di tipo Punto2D.
//costruttori:
      public Segmento(Punto2D inizio, Punto2D fine) throws DuplicatePointException
      //EFFECTS: if inizio.equals(fine) throw DuplicatePointException
      //     else costruisce this come segmento di due punti: (inizio, fine)
//observers:
      public float lunghezza() {
      //EFFECTS: restituisce la lunghezza di this
      }
      public Punto2D puntoIniziale()
      //EFFECTS: restituisce l'estremo iniziale di this
      public Punto2D puntoFinale()
      //EFFECTS: restituisce l'estremo finale di this
      }
}
public class Spezzata {
//OVERVIEW: una spezzata è un insieme ordinato di almeno 2 punti di tipo
// Punto2D, tutti distinti.
// Una spezzata tipica è [p1, p2, ..., pn], corrispondente alla linea spezzata che unisce
//p1, p2, ..., pn nell'ordine. Tipo mutabile.
//costruttori
      public Spezzata(Punto2D inizio, Punto2D fine) throws DuplicatePointException
      //EFFECTS: se Inizio != Fine restituisce una spezzata di due punti:
      //(inizio, fine), altrimenti throw DuplicatePointException
//observers:
      public Punto2D puntoIniziale (){}
      //EFFECTS: restituisce il punto di partenza p1 di this
      public Punto2D puntoFinale(){}
      //EFFECTS: restituisce il punto di arrivo pn di this
      public float lunghezza() {}
      //EFFECTS: restituisce la lunghezza complessiva di this
      public boolean puntoIncluso(Punto2D p) {}
      //EFFECTS: ritorna true se p è equals a un punto di this, altrimenti false.
//modifiers:
      public void addInizio(Punto2D p){} throws DuplicatePointException;
      //EFFECTS: this_post = [p, p1, .., pn]: aggiunge p a this all'inizio (se p !=pi).
public void addFine (Punto2D p){} throws DuplicatePointException;
      //EFFECTS: this_post = [p1, .., pn, p]: aggiunge p a this in fondo (se p !=pi).
 }
```

(l'esercizio continua sulla pagina successiva)

(continua l'Esercizio 3).....

Completare l'implementazione parziale della classe Spezzata descritta qui, in cui si usa come struttura dati un Vector di *Segmento*: il primo elemento del Vector è il segmento che unisce p1 a p2; il secondo è il segmento che unisce p2 a p3, ecc. Completare la classe con:

a) i modificatori di visibilità e la dichiarazione delle variabili.
b) il codice del costruttore e dei metodi (salvo che per il metodi puntoIncluso(), che non è riportato).
c) AF e RI, senza implementarli.

Infine, verificare che RI è effettivamente un invariante per Spezzata.
Si noti che non è necessario invece implementare la classe Segmento.

NB: la classe Vector del package java.util ha i seguenti metodi, di cui alcuni possono essere utili nell'implementazione di Spezzata:
public Object get(int i) restituisce elemento in posizione i
public Object elementAt(int i): come Object get(int i)
public Object lastElement(): restituisce ultimo elemento del Vector
public Object firstElement(): restituisce primo elemento del Vector
public Iterator iterator(): restituisce generatore agli elementi del Vector, in ordine.
public int size(): restituisce il numero di elementi del Vector
public void add(Object o): inserisce o come nuovo elemento alla fine del Vector
public void add(int i, Object o): inserisce o come nuovo elemento in posizione i del Vector, shiftando opportunamente gli elementi a destra

```
import java.util.* ;
public class Spezzata {
//OVERVIEW (v. specifica)
//AF(c) =
.....
.....
```
[segmenti.firstElement.puntoIniziale() segmenti.get(0).puntoFinale(), segmenti.get(1).puntoFinale(), ... , segmenti.lastElement().puntoFinale]

//attenzione: il Vector nel rep contiene dei segmenti, non dei punti, ma la specifica astratta è composta solo da punti!!!

//RI(c) =
segmenti != null && segmenti.size() >0 &&
for all i, 0<= i <segmenti.size(), segmenti.get(i) è un Segmento &&
for all i, 0<= i <segmenti.size()-1,
segmenti.get(i).puntoFinale() == segmenti.get(i+1).puntoIniziale() &&
for all i,j, 0<= i<j <segmenti.size()-1,
segmenti.get(i).puntoFinale()!=segmenti.get(j).puntoFinale() &&

lung = $\sum_{i=0}^{segmenti.size()-1}$ **segmenti.get(i).lunghezza()**

NB: la condizione che tutti i punti finali sono distinti può anche essere omessa, in quanto implicata dalla specifica della classe......
.....
.....
.....
//rep:
private..... Vector segmenti;

private..... float lung;
//costruttore:
public Spezzata(Punto2D inizio, Punto2D fine) throws DuplicatePointException{
Segmento s = new Segmento(inizio, fine); .....
//costruttore di Segmento lancia già eccezione...
segmenti = new Vector(1); .....
//il Vector segmenti deve essere inizializzato!!!
segmenti.add(s); .....
lung = s.lunghezza(); .....**// anche lung deve essere inizializzata!**
}
//observers:
public Punto2D puntoIniziale (){
..... **return ((Segmento) segmenti.firstElement()).puntoIniziale();**
//attenzione: il Vector contiene dei segmenti, non dei punti!!!
}
public Punto2D puntoFinale(){
..... **return ((Segmento) segmenti.lastElement()).puntoFinale();**
}
public float lunghezza () {return lung;} **(l'esercizio continua sulla pagina successiva)**

NB: il senso inteso di "lunghezza" di una spezzata è quello di somma delle lunghezze dei segmenti che la compongono (la lunghezza di un segmento è la distanza fra i suoi due punti estremi). Si considera comunque accettabile anche la soluzione in cui la lunghezza di una spezzata è pari al numero di segmenti di cui è composta, anche se questa interpretazione è meno naturale.

(continua l'Esercizio 3).....

```
//Modifiers:
    public void addInizio(Punto2D p) throws DuplicatePointException {
        if (puntoIncluso(p)) throw new  DuplicatePointException("Segmenti.addInizio");
    //inserisce il nuovo segmento all'inizio del Vector:
        Segmento s = new Segmento(p, puntoIniziale());
        segmenti.add(0,s);
    ……lung = lung + s.lunghezza(); //aggiornare lung!!!
    }
    public void addFine (Punto2D p) throws DuplicatePointException {
        if (puntoIncluso(p)) throw new  DuplicatePointException("Segmenti.addFine");
        //inserisce il nuovo segmento alla fine del Vector:
        Segmento s = new Segmento(puntoFinale(),p);
        segmenti.add(s);
        lung = lung + s.lunghezza();
    ..........
    ..........
    ..........
    ..........
    }
    }
}
```

Prova che RI è invariante:

1) **il costruttore Spezzata inizializza segmenti con il segmento passato come argomento, e aggiorna la lunghezza lung;**
2) **Il modificatore addInizio:**
    a. **aggiunge un segmento solo se il suo punto iniziale non è già presente (test con puntoIncluso());**
    b. **mantiene la proprietà che il puntoFinale di ogni segmento è equals puntoIniziale del segmento successivo;**
    c. **aggiorna la lunghezza lung.**
3) **Idem per addFine.**

Non ci sono altri metodi che possono modificare il rep.

Esercizio 4

a) Supporre che la classe Spezzata dell'esercio 3 fornisca un metodo iteratore punti() che restituisce un generatore per la classe Spezzata. Il generatore restituisce i punti di una Spezzata nello stesso ordine in cui i punti vi sono presenti. Utilizzare l'iteratore così definito per implementare il metodo puntoIncluso() della classe Spezzata, supponendo di usare per il confronto fra punti il metodo public boolean equals(Punto2D p) definito in Punto2D.
b) Implementare il metodo iteratore punti() e la classe del generatore, senza il metodo remove().

```
class Spezzata ....{
  public boolean puntoIncluso(Punto2D p) {
//NB: non è necessario, ma non è errato, fare riflessione o try...catch di
//NoSuchElementException, che è unchecked e può essere lanciata da next().
     //EFFECTS: ritorna true se p è equals a un punto di this, altrimenti false.
  for (Iterator i = punti(); i.hasNext();)
    if (p.equals((Punto2D) i.next())) return true;
//NB: serve cast perché Punto2D non ha ridefinito equals(Object)
//è corretto anche scrivere:  ((Punto2D) i.next()).equals(p)
return false;
}

public Iterator punti() { return new PuntiIter (this);}

 private static class PuntiIter implements Iterator {
        int posizione;
        Vector elementi;
    PuntiIter(Spezzata s) { posizione = -1; elementi = s.segmenti;}

     public Object next() throws NoSuchElementException {
      if (posizione >= elementi.size()) throw new NoSuchElementException();
//il throw qui sopra è previsto dalla specifica di next()
      if (posizione++ == -1) return
(Segmento)elementi.firstElement()).puntoIniziale();
      return ((Segmento)elementi.get(posizione)).puntoFinale();
      }

     public boolean hasNext() { return (posizione < elementi.size());}
     public void remove() {throw new UnsupportedOperationException()}
    //NB:  la remove poteva essere omessa.
 }
}//class Spezzata
```

Si osservi che la soluzione qui illustrata è solo un esempio fra le tante possibili, più o meno semplici. L'importante è che:

1) **next() restituisca i *punti*, e non i segmenti, componenti una spezzata. Non ci devono essere ripetizioni (p.es. restituisca tutti i punti iniziali e l'ultimo punto finale)**
2) **next() lanci l'eccezione NoSuchElementException quando non ci sono piu' punti.**